assignment3_part1

October 23, 2026

1 Assignment 3: Mining Vectors and Matrices (Part I)

Welcome to Assignment 3! In this assigment, you will be playing with vectors and matrices. Similar to itemsets, or other representations of data, we care about *similarity* and *patterns*, the two basic outputs of data mining and building blocks of advanced functionalities. Through this assignment, you will get hands-on experience to derive both similarity measurements and patterns with various vector and matrix operations.

We have curated another real-world dataset for you – restaurant ratings on Yelp. The original data set can be found here. To make this assignment more trackable, we have preprocessed the data and only kept the restaurants in Montréal, Canada. As with all real-world problems, be sure to sanity-check the quality of the data and be ready to handle "unexpected" scenarios of the "Wild Thing."

In this assignment, you will: * Represent the dataset as a matrix and check the row and column vectors. * Implement various vector similarity/distance measures. * Find the top similar vectors to a given vector. * Compute SVD transformation of the Matrix/Vectors and analyze the vectors with reduced dimensionality.

In Part I of the assignment, we will load the dataset, transform it into vector/matrix format, and perform necessary sanity checks. You only need to code very little in this part. Please run all the code blocks in order and read the descriptions carefully.

First, let's import the packages and dependencies that will be used later.

[1]:
```
import pandas as pd
import numpy as np
```

1.1 Data Preprocessing

Let's start by loading the data and preview the first few lines. We will use two data files. The montreal_business.csv file stores the attributes of the restaurants, and the montreal_rating.csv file stores the user ratings of the restaurants. Both businesses and users are assigned randomized string identifiers (business_id and user_id), as a common way to preserve the privacy of customers.

[2]:
```
business_df = pd.read_csv('assets/montreal_business.csv')
business_df.head()
```

[2]:
```
              business_id                name  \
0  -1xuC540Nycht_iWFeJ-dw             Romados
```

```
1  -7bRnaHp7OHz8KW-THqP4w  Buffalo Bill Wings
2  -92cC6-X87HQ1DE1UHOx3w              Junior
3  -AgfhwHOYrsPKt-_xV_Ipg           Soupesoup
4  -FDkvLmwaBrtVgYFqEWeWA    Café Juicy Lotus

                                      categories  stars postal_code
0  Restaurants, Portuguese, Bakeries, Food, Chick…    4.5     H2W 1C8
1                          Restaurants, Chicken Wings    1.5     H4C 1G7
2     Pan Asian, Restaurants, Asian Fusion, Filipino    3.5     H3J 1M8
3                Sandwiches, Restaurants, Cafes, Soup    4.0     H2W 1G8
4  Vegetarian, Food, Salad, Restaurants, Organic …    3.5     H4A 1C9
```

[3]:
```
business_df.set_index('business_id', inplace=True)
business_df.head()
```

[3]:
```
                                    name  \
business_id
-1xuC540Nycht_iWFeJ-dw             Romados
-7bRnaHp7OHz8KW-THqP4w  Buffalo Bill Wings
-92cC6-X87HQ1DE1UHOx3w              Junior
-AgfhwHOYrsPKt-_xV_Ipg           Soupesoup
-FDkvLmwaBrtVgYFqEWeWA    Café Juicy Lotus

                                                        categories  \
business_id
-1xuC540Nycht_iWFeJ-dw  Restaurants, Portuguese, Bakeries, Food, Chick…
-7bRnaHp7OHz8KW-THqP4w                          Restaurants, Chicken Wings
-92cC6-X87HQ1DE1UHOx3w     Pan Asian, Restaurants, Asian Fusion, Filipino
-AgfhwHOYrsPKt-_xV_Ipg                Sandwiches, Restaurants, Cafes, Soup
-FDkvLmwaBrtVgYFqEWeWA  Vegetarian, Food, Salad, Restaurants, Organic …

                        stars postal_code
business_id
-1xuC540Nycht_iWFeJ-dw    4.5     H2W 1C8
-7bRnaHp7OHz8KW-THqP4w    1.5     H4C 1G7
-92cC6-X87HQ1DE1UHOx3w    3.5     H3J 1M8
-AgfhwHOYrsPKt-_xV_Ipg    4.0     H2W 1G8
-FDkvLmwaBrtVgYFqEWeWA    3.5     H4A 1C9
```

Every row is a restaurant, and the "stars" field stores the aggregated rating of all its customers. Other fields should be self-explanatory.

[4]:
```
review_df = pd.read_csv('assets/montreal_user.csv')
review_df.head()
```

[4]:
```
              business_id                 user_id  stars
0  i_oghUPqLzzJtxC6Zm-D2A  -8nmj3B-tfY_vFiimtBOsw    5.0
```

```
1  i_oghUPqLzzJtxC6Zm-D2A  11V_zUrpadPKvPQpQ-Ctiw    4.0
2  i_oghUPqLzzJtxC6Zm-D2A  3AqRjAvCdBiM6s0f58A5eA    5.0
3  i_oghUPqLzzJtxC6Zm-D2A  LN4ajhAiIshKkXJQhuh6uA    4.0
4  i_oghUPqLzzJtxC6Zm-D2A  T6_H4brErsgxmUEvv9VLOg    3.0
```

Every row in this user-rating data file corresponds to a restaurant-user pair, and the "stars" column stores the individual (unaggregated) score that users gave to that restaurant.

1.1.1 Constructing restaurant-user matrix:

The first thing is to transform the user-restaurant ratings into a matrix. Each row vector represent the ratings of one restaurant and each dimension of the row vector (aka. a column) represents a user. Therefore, the j-th dimension of the i-th row vector represents the rating of user j on the restaurant i. We can perform pivoting on the `review_df` dataframe to generate such a matrix, and then view the first few row vectors.

[5]:
```
rating_df = review_df.pivot_table(index=['business_id'], columns=['user_id'],␣
 ↪values='stars')
rating_df.head(5)
```

[5]:
```
user_id                --BumyUHiO_7YsHurb9Hkw  --NIc98RMssgyOmSZL3vpA  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                -0zUpn_6kTWXHWzB85u7yA  -1MF2tosrw2WcCxeVNk81Q  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                -1wbglcr6x1qrUbqP1YAIA  -3gey5u7j0qj5SZNrOHIQg  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                -3s52C4zL_DHRK0ULG6qtg  -3uRUjmr1uEn-lLrhwRMCg  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
```

```
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                -4pYtFJbZYzKqmiOjGr4ag  -4tyD87vW73HGOmQlXkOuw  …  \
business_id                                                            …
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN  …
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN  …
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN  …
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN  …
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN  …

user_id                zwOiJvD09N1dNf-ts7wscw  zwsVQwyxkz-s4rvBCaPyZw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                zyLIp7DRFqeIE1Grrh3Zng  zyYWUdaodHOh1jCZAvFRPg  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                zyaQNsI73cLOKttvGnj4bw  zyg4-MFtfPWmwucVazSjfw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                zzKsgIF472IyDpJxmHDIpw  zzgb9L3NBT9V8BGw_DcVYw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
-7bRnaHp7OHz8KW-THqP4w                    NaN                     NaN
-92cC6-X87HQ1DE1UHOx3w                    NaN                     NaN
-AgfhwHOYrsPKt-_xV_Ipg                    NaN                     NaN
-FDkvLmwaBrtVgYFqEWeWA                    NaN                     NaN

user_id                zznMmC74DNBcBG62gj3guA  zzo--VpSQh8PpsGVeMC1dQ
business_id
-1xuC540Nycht_iWFeJ-dw                    NaN                     NaN
```

```
-7bRnaHp7OHz8KW-THqP4w                    NaN                    NaN
-92cC6-X87HQ1DE1UHOx3w                   NaN                    NaN
-AgfhwHOYrsPKt-_xV_Ipg                   NaN                    NaN
-FDkvLmwaBrtVgYFqEWeWA                   NaN                    NaN

[5 rows x 11937 columns]
```

However, directly using this matrix in production might not be a good practice. The matrix can be huge in size and yet very very sparse, as not every user has rated every restaurant. In other words, most of the cells of the matrix have missing values (NaN).

Let's briefly examine how sparse the matrix is.

[6]:
```
n_entry = review_df.shape[0]
n_business = review_df.business_id.unique().size
n_user = review_df.user_id.unique().size
print("total entry:", n_entry)
print("# business:", n_business)
print("# user:", n_user)
print(f"density:{n_entry / (n_business * n_user):.4f}")
```

```
total entry: 71741
# business: 2779
# user: 11938
density:0.0022
```

Before any analysis, we also need to perform sanity checks. For matrix data, the first thing we always do is the check the dimensionality.

1.1.2 Exercise 1. Check the dimensionality (5 pts)

Complete the `row_col_count` function below to return the numbers of rows and columns of the rating matrix.

Hint: the `.shape` attribute of a dataframe returns the number of rows and columns in a tuple.

[7]:
```
def row_col_count(rating_df):

    n_row = 0
    n_col = 0

    # YOUR CODE HERE
    #raise NotImplementedError()
    n_row, n_col = rating_df.shape

    return n_row, n_col
```

[8]:
```
n_row, n_col = row_col_count(rating_df)
assert n_row == 2770
```

```
assert n_col == 11937
```

[9]:
```
rating_df.shape
```

[9]:
```
(2770, 11937)
```

Comparing the numbers with the earlier cells, we have 2779 businesses in review_df, but we only have 2770 rows in rating_df. What happened? Let's further examine the difference.

[10]:
```
set(business_df.index) - set(rating_df.index)
```

[10]:
```
{'3mr1-uwrYFwFDQO9WNEVqw',
 '7RvCrO6n2jkC8IQvfdm04g',
 'O7uzVmRB8GDMU8XaX5l7Eg',
 'RW4wwNBB-TDJcN2vCsLMRw',
 'VTb9w9PowA76vhBVgrwsIw',
 'ZByHOUsGw7q5QC-EsJfmIg',
 'eC3LV3CpXPoB4kpMDIbCVQ',
 'nK4kWsScyDBHdcxtfszzcQ',
 'xFxlyQPT27xc2dsqyzQKUg'}
```

We see that difference of 9, (2779 - 2770), comes solely from 9 businesses missing in the `rating_df`. Do they exist in `review_df`?

[11]:
```
missing_business_id = set(business_df.index) - set(rating_df.index)
review_df[review_df.business_id.isin(missing_business_id)]
```

[11]:

	business_id	user_id	stars
8943	O7uzVmRB8GDMU8XaX5l7Eg	NaN	NaN
30631	xFxlyQPT27xc2dsqyzQKUg	NaN	NaN
43765	VTb9w9PowA76vhBVgrwsIw	NaN	NaN
44263	eC3LV3CpXPoB4kpMDIbCVQ	NaN	NaN
45411	3mr1-uwrYFwFDQO9WNEVqw	NaN	NaN
46355	ZByHOUsGw7q5QC-EsJfmIg	NaN	NaN
57748	7RvCrO6n2jkC8IQvfdm04g	NaN	NaN
61904	RW4wwNBB-TDJcN2vCsLMRw	NaN	NaN
70186	nK4kWsScyDBHdcxtfszzcQ	NaN	NaN

It is now clear that the 9 missing businesses do not have any valid entries in the `review_df`, so they do not have corresponding rows in `rating_df` after pivoting. To be consistent, we could create 9 additional rows in `rating_df` with all NaN values. Or, we can simply remove the businesses from `business_df` to make the two dataframes coherent.

[12]:
```
business_df.drop(missing_business_id, inplace=True)
```

There are several ways of dealing with missing values in a matrix (the NaN cells in the pivoted dataframe). For now, let's simply fill the NaN cells with 0. This may not be a good practice in reality. Not only will we be dealing with a much larger (denser) dataset, which requires a lot more

computing resources, but more importantly, we are implicitly making an assumption that the users are rating those restaurants with 0 star. Do they really dislike those restaurant or have they not been to them? Can you think of a more reasonable method to fill in NAs?

[13]:
```
rating_df.fillna(0, inplace=True)
rating_df.head(5)
```

[13]:
```
user_id                --BumyUHiO_7YsHurb9Hkw  --NIc98RMssgy0mSZL3vpA  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -0zUpn_6kTWXHWzB85u7yA  -1MF2tosrw2WcCxeVNk81Q  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -1wbglcr6x1qrUbqP1YAIA  -3gey5u7j0qj5SZNrOHIQg  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -3s52C4zL_DHRK0ULG6qtg  -3uRUjmr1uEn-lLrhwRMCg  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -4pYtFJbZYzKqmiOjGr4ag  -4tyD87vW73HGOmQlXk0uw  …  \
business_id                                                            …
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0  …
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0  …
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0  …
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0  …
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0  …
```

```
user_id                zwOiJvD09N1dNf-ts7wscw  zwsVQwyxkz-s4rvBCaPyZw  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                zyLIp7DRFqeIE1Grrh3Zng  zyYWUdaodHOh1jCZAvFRPg  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                zyaQNsI73cLOKttvGnj4bw  zyg4-MFtfPWmwucVazSjfw  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                zzKsgIF472IyDpJxmHDIpw  zzgb9L3NBT9V8BGw_DcVYw  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                zznMmC74DNBcBG62gj3guA  zzo--VpSQh8PpsGVeMC1dQ
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

[5 rows x 11937 columns]
```

Now we should obtain a restaurant x user matrix fully filled with ratings. This concludes the first part of the assignment.